

Data Platform Case Study

1. Context

We receive our customers' financial lives through Brazil's Open Finance protocol. Two raw feeds land continuously — **positions**, which are balance snapshots of a customer's investments and cash, and **transactions**, the buys, sells, redemptions, income and transfers behind those investments. A batch job reads the raw records every few minutes; this is incremental batch, not streaming.

Almost everything the product shows is derived from those two feeds: the wealth page, monthly cash flow, net worth over time, portfolio evaluation. Each is a different team's question at a different grain, and today each is answered by a pipeline written from scratch against the raw records — which scales neither in compute nor in the number of people who can say what a number means. Your task is to build the layer in between: canonical tables with explicit contracts, an incremental job that maintains them, and enough guarantees that another team can build a new consumer without talking to you first.

2. Inputs

2.1 Datasets provided

Two files, patterned on production data, covering a few hundred synthetic customers:

File	Grain	Contents
<code>raw_positions.parquet</code>	one row per record, as delivered	The institution's payload verbatim, plus delivery metadata: institution, customer, account, record and delivery identifiers, arrival time. Snapshot cadence is irregular and differs by institution.
<code>raw_transactions.parquet</code>	one row per record, as delivered	The same delivery metadata, for the movements on those investments. The payload's own fields vary by product.

Fields and enumerations are those of the Open Finance Brasil API. The specification is public, and reading it is part of the work: <https://openfinancebrasil.atlassian.net/wiki/spaces/OF/overview>. An institution respecting it is a hope, not a guarantee.

2.2 What the pipeline has to survive

Our quality ledger sorts provider defects into two tiers.

Within a record — a required field arriving empty; a value carrying the right concept in the wrong form, such as a tax identifier with a decimal tail; a legal value outside the enumeration; a field valid on its own that contradicts another in the same payload.

Across records — the same holding delivered twice in one sync under two identifiers; the same holding reappearing later under a new identifier, so the old one freezes and the portfolio double-counts; a holding valued at zero in one sync and back above zero in the next with its quantity unchanged.

The second tier is the hard one: every record passes every per-record rule, and the defect exists only in the comparison. Decide what you reject, what you quarantine and what you admit with a warning — and how a consumer finds out which happened.

2.3 AI tools

We use AI tooling heavily ourselves and expect you to. How you directed it is fair game in the walkthrough.

3. Deliverables

1. **A layered data model** — raw → canonical → consumption, with schema changes applied through versioned, replayable steps. Any store is acceptable provided the choice is justified. State the physical layout — partitioning, clustering, file sizing — and the query pattern it serves.
 2. **An incremental job** with an explicit watermark. Running it twice over the same window leaves the same result as running it once. Cover late records, a record re-delivered with a different value, and reprocessing history after you fix a transform.
 3. **An output contract, as an executable artifact** — YAML, JSON Schema, Pydantic, dbt `schema.yml`; your choice.
 4. **A page, under consumers/** — `wealth`: what one customer holds, and the movements behind it. A SQL query or a dbt model over the canonical layer, never raw. Commit its output for a few sample customers.
-

4. Constraints and expectations

- **Scale** — your implementation may run locally, but your video must describe a credible path to production. The sample is small; at real volume the canonical layer must absorb millions of records a day across thousands of customers and dozens of institutions, and serve concurrent queries from hundreds of analysts and services.
 - **Reproducibility** — we must be able to clone and run end-to-end from a clean environment.
 - **Compliance** — the sample is synthetic, but in production every field is personal financial data. Design for the boundaries that would apply if it were real.
 - **Scope** — a smaller surface, well argued, beats a complete one badly resolved.
-

5. Evaluation criteria

5.1 Correctness

Does the `wealth` output hold up? Is each holding read at the right grain, valued at the right moment, and free of the duplicates the feed delivers?

5.2 Modeling judgment

Are the layer boundaries in the right place? Does the canonical layer hold positions and movements at the grain each is delivered, with the provider defects resolved once rather than again in every consumer?

5.3 Exposability

Can another team add a consumer, or another engineer a source, without you in the loop?

5.4 Operability

Does the pipeline tell you when it is wrong? Is there a freshness signal, a way to backfill, and a way to replay history after a fix?

5.5 Cost

Does cost grow with the number of customers connected through Open Finance, or slower than that? Do the partitioning, clustering, file layout and compute sizing each have a reason, and does the design hold at ten times the customers?

6. Submission instructions

- A **GitHub repository** with your code, configuration, and a `README.md` describing how to run it end-to-end. State what you deliberately left out and why.
- `contracts/` — your executable contract artifacts.
- `consumers/` — the consumer, together with its output.
- A **short video walkthrough** (≤ 5 minutes) covering your design, how a new consumer lands on your platform, and the one decision you are least sure about.

Deadline: Monday 31 August 2026, 20:00 BRT.